IMPERIAL

pyop3:
An Execution Abstraction for Compact Computational Kernels on Unstructured Meshes

Connor J. Ward
David A. Ham
Jack D. Betteridge

c.ward20@imperial.ac.uk

1. Motivation

* Numerically solving PDEs over unstructured meshes requires one to loop over mesh entities and perform ”local” computations in regions. * For example, in the finite element method, ...

* The variables in the system of equations (the DoFs) are associated with the different mesh entities. So when doing these loops one needs to express both the iteration regime (?) as well as the indirections from cells (e.g.) to supported DoFs.

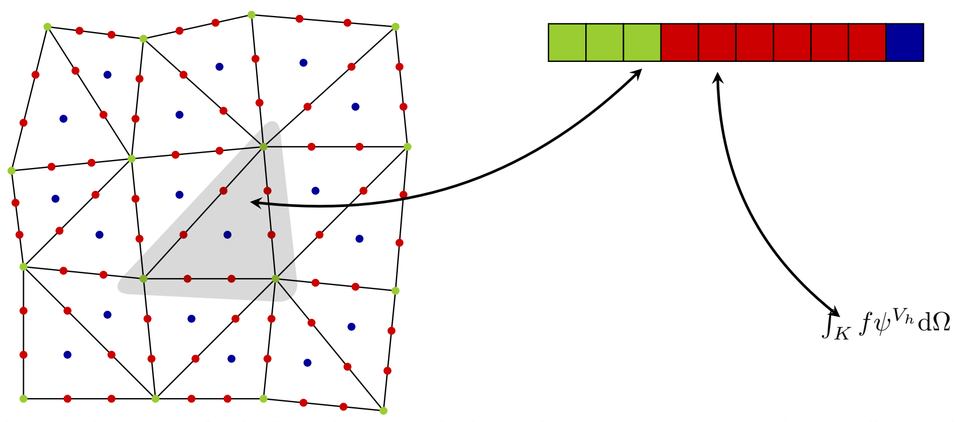

Figure 1: The assembly process for the finite element method. DoFs local to a cell are collected in a small array which is then used to perform the cell-wise computation with the result being scattered back to a global vector or matrix.

"With great composability comes great (expressive) power"

A number of libraries exist that already provide this abstraction. However they all suffer from either: (1) having a custom mesh abstraction, and hence being unable to use mature existing packages (e.g. [2]), or (2) having no concept of a mesh altogether, which hinders composability (e.g. [3]).

PYTHON

```
loop(cell := mesh.cells.index(),
     kernel(vec1[cell_to_vertex(cell)], vec2[cell]))
```

CODE GENERATION

```
void pyop3_kernel(array_0, array_1, array_2) {
  double t_0[3];
  double t_1[1];
  for (int32_t i_0 = 0; i_0 < ncells; ++i_0) {
    for (int32_t i_1 = 0; i_1 < 3; ++i_1)
      t_0[i_1] = array_0[array_1[3 * i_0 + i_1]];
    t_1[0] = 0.0;
    kernel(&(t_0[0]), &(t_1[0]));
    array_2[i_0] = array_2[i_0] + t_1[0];
  }
}
```

C

2. Introducing pyop3

To bridge this divide, we introduce pyop3, a new abstraction for applying local computations to a mesh. The design of pyop3 is inspired by its precursor, PyOP2 [3], and PETSc [1], a popular PDE toolkit. Powered by a novel abstraction for representing mesh data, pyop3 can generate efficient code in just a handful of lines of Python code (Figure **??**).

3. A new abstraction for mesh data

To reconcile the need for composability with the need to be able to generate low-level code, pyop3 introduces a new abstraction for representing mesh data. From the reference elements in Figure 2 and the data layout in Figure 3 it is clear that (a) there is some structure present, and (b) this structure is not sufficiently uniform to be represented as an N-dimensional array. pyop3 generalises N-dimensional arrays to mesh data structures by representing them as trees.

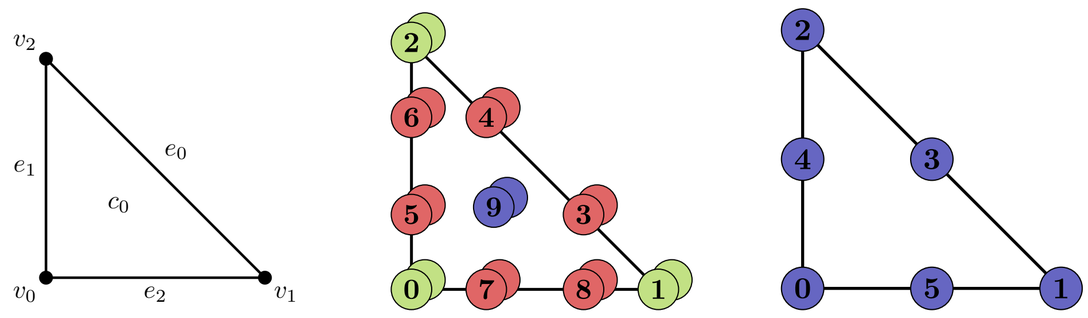

Figure 2: The Scott-Vogelius finite element pair (middle and right) for the reference triangle (left). DoFs associated with vertices, edges and cells are shown in green, red and blue respectively. The lower-order element (right) is discontinuous, so all the DoFs are considered to belong to the cell.

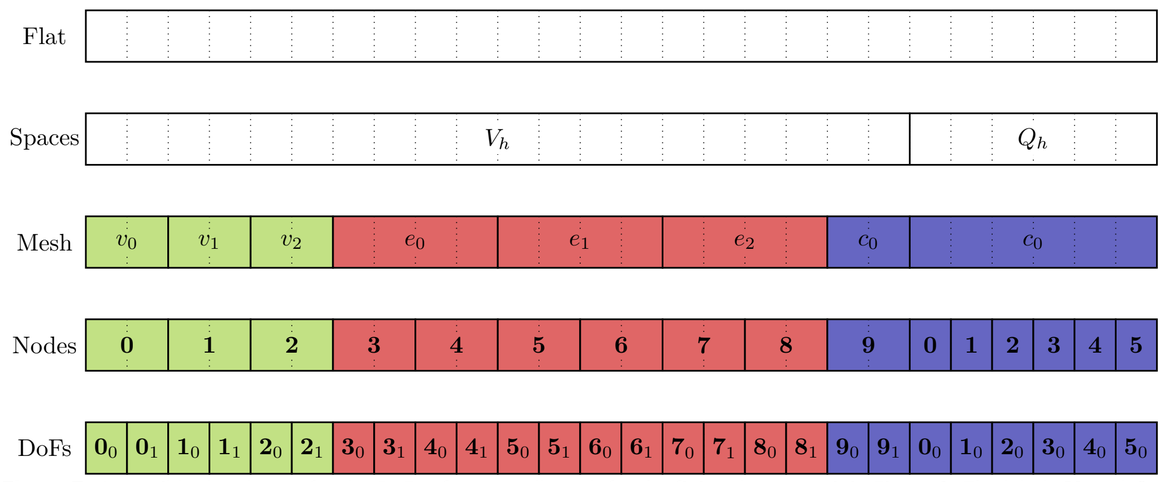

Figure 3: A possible data layout for the DoFs of a one-cell mesh for the finite element pair in Figure 2. The labels V_h and Q_h represent the function spaces of the two elements.

4. Future work

References

[1] Satish Balay, Shrirang Abhyankar, et al. PETSc Users Manual. ANL-95/11 - Revision 3.15. Argonne National Laboratory, 2021.

[2] Fredrik Kjolstad, Shoaib Kamil, et al. “Simit: A Language for Physical Simulation”. In: 35.2 (25, 2016). (Visited on 03/29/2022).

[3] Florian Rathgeber, Graham R. Markall, et al. “PyOP2: A High-Level Framework for Performance-Portable Simulations on Unstructured Meshes”. In: 2012 SC Companion: High Performance Computing, Networking Storage and Analysis. 2012. (Visited on 04/19/2020).